

What is a File?

A file is a unit used to store data permanently on a computer.

Whenever we save information such as text, images, videos, programs, or documents on a computer, it is stored in the form of a file.

Simple Explanation:

We use a notebook to write and store information.

Similarly, a file is a digital notebook that stores information on a computer.

Exam Definition

"A file is a collection of related data stored permanently on a storage device and identified by a unique name."

Examples of Files

Student.txt

Marks.xlsx

Photo.jpg

Video.mp4

Program.py

These files store different types of information such as text, marks, images, videos, and programs.

Need for File Handling

File Handling is used to store, retrieve, and manage data permanently in a computer.

Why Do We Need File Handling?

1. Permanent Storage of Data

Data stored in variables is lost when the program terminates. File handling helps store data permanently for future use.

2. Data Retrieval

Stored data can be accessed and used whenever required.

3. Handling Large Amounts of Data

Files can store a large volume of data efficiently, which is difficult to manage using variables alone.

4. Data Sharing

Files make it easy to share information between different programs and users.

5. Data Backup and Record Keeping

Important information such as student records, employee details, and transaction data can be maintained safely.

6. Reduces Repeated Data Entry

Once data is stored in a file, it can be reused without entering it again.

7. Supports Real-World Applications

Most applications such as banking systems, college management systems, and hospital management systems use files to store data.

Simple Definition

"File handling is needed to store data permanently and retrieve it whenever required."

Exam Point of View

Need for File Handling:

1. Permanent storage of data.
2. Easy retrieval of information.
3. Handling large amounts of data.
4. Data sharing between applications.

5. Backup and maintenance of records.
 6. Reduces repeated data entry.
 7. Supports real-world applications.
-

TYPES OF FILES IN PYTHON

Introduction

In Python, files are mainly classified into two types:

1. Text Files
 2. Binary Files
-

1. TEXT FILES

Definition

A text file stores data in the form of characters (letters, numbers, and special symbols) that can be read directly by humans.

Examples

- .txt
- .csv
- .html
- .xml
- .log

Characteristics of Text Files

- Data is stored as characters.
- Human-readable format.
- Can be opened using Notepad or any text editor.
- Easy to create and modify.
- Uses more storage space compared to binary files.
- Slower processing for large amounts of data.

Advantages of Text Files

1. Easy to read and understand.
2. Easy to edit using text editors.

3. Portable across different operating systems.
4. Suitable for storing textual information.
5. Debugging is easy.

Disadvantages of Text Files

1. Requires more storage space.
2. Slower read and write operations.
3. Less secure because data is visible.
4. Not suitable for multimedia data.
5. Inefficient for large datasets.

Example

```
file = open("student.txt", "w")  
file.write("Rohit")  
file.close()
```

2. BINARY FILES

Definition

A binary file stores data in binary format (0s and 1s) and cannot be read directly by humans.

Examples

- .dat
- .bin
- .exe
- .jpg
- .png
- .mp3
- .mp4
- .pdf

Characteristics of Binary Files

- Data is stored in binary form.
- Not human-readable.

- Requires special software or programs to interpret.
- Occupies less storage space.
- Faster processing than text files.
- Suitable for storing complex and multimedia data.

Advantages of Binary Files

1. Faster read and write operations.
2. Uses less storage space.
3. Better performance for large data.
4. More secure than text files.
5. Suitable for images, audio, video, and complex objects.

Disadvantages of Binary Files

1. Cannot be read directly by humans.
2. Difficult to edit manually.
3. Requires specific programs for viewing.
4. Debugging is difficult.
5. Data corruption may be harder to recover.

Example

```
file = open("data.dat", "wb")
file.write(b"Python")
file.close()
```

COMPARISON BETWEEN TEXT FILES AND BINARY FILES

Feature	Text File	Binary File
Data Format	Characters	Binary Data (0s and 1s)
Human Readable	Yes	No
Storage Space	More	Less
Processing Speed	Slower	Faster
Editing	Easy	Difficult

Feature	Text File	Binary File
Security	Less	More
Examples	.txt, .csv	.jpg, .mp3, .dat

EXAM DEFINITIONS

Text File

"A text file is a file that stores data in the form of characters and can be read directly by humans."

Binary File

"A binary file is a file that stores data in binary format and cannot be read directly by humans."

CONCLUSION

Python supports Text Files for storing readable textual data and Binary Files for storing efficient, high-performance, and multimedia data. The choice of file type depends on the nature of the data and the requirements of the application.

FILE OPERATIONS IN PYTHON

Introduction

File operations are used to create, read, write, append, and close files. Python provides built-in functions to perform these operations easily.

1. OPENING A FILE

Definition

Before performing any operation on a file, it must be opened using the `open()` function.

Syntax

```
file = open("filename", "mode")
```

Example

```
file = open("student.txt", "r")
```

Explanation

- student.txt is the file name.
 - "r" means read mode.
 - The file is opened for reading.
-

2. WRITING TO A FILE

Definition

Writing is used to store data into a file.

Example

```
file = open("student.txt", "w")
```

```
file.write("Rohit")
```

```
file.close()
```

Explanation

Step 1

```
file = open("student.txt", "w")
```

- Opens the file in write mode.
- If the file does not exist, Python creates it.
- If the file already exists, old data will be erased.

Step 2

```
file.write("Rohit")
```

- Writes the text "Rohit" into the file.

Step 3

```
file.close()
```

- Closes the file.
- Saves the data permanently.

Output in File

Rohit

3. READING A FILE

Definition

Reading is used to retrieve data from a file.

Example

```
file = open("student.txt", "r")
data = file.read()
print(data)
file.close()
```

Explanation

Step 1

```
file = open("student.txt", "r")
```

- Opens the file in read mode.

Step 2

```
data = file.read()
```

- Reads all contents from the file.

Step 3

```
print(data)
```

- Displays file contents on the screen.

Output

Rohit

4. APPENDING DATA TO A FILE

Definition

Appending is used to add new data at the end of an existing file without deleting old data.

Example

```
file = open("student.txt", "a")
file.write("\nKumar")
file.close()
```


Explanation

Step 1

```
file = open("student.txt", "a")
```

- Opens the file in append mode.

Step 2

```
file.write("\nKumar")
```

- Adds "Kumar" at the end of the file.

Step 3

```
file.close()
```

- Closes the file.

Output in File

Rohit

Kumar

5. READING A FILE LINE BY LINE

Example

```
file = open("student.txt", "r")
```

```
for line in file:
```

```
    print(line)
```

```
file.close()
```

Explanation

- Reads one line at a time.
- Useful when files contain large amounts of data.
- Memory usage is reduced.

Output

Rohit

Kumar

6. CLOSING A FILE

Definition

After completing file operations, the file should be closed.

Example

```
file = open("student.txt", "r")  
file.close()
```

Explanation

- Releases system resources.
- Prevents data loss.
- Good programming practice.

COMMON FILE MODES

Mode Description

r	Read mode
w	Write mode
a	Append mode
r+	Read and Write
rb	Read Binary
wb	Write Binary
ab	Append Binary

SUMMARY OF FILE OPERATIONS

1. Open a file using `open()`.
2. Read data using `read()`.
3. Write data using `write()`.
4. Append data using `write()` in append mode.
5. Close the file using `close()`.

EXAM DEFINITIONS

File Open

Opening a file means establishing a connection between the program and the file for performing operations.

File Read

Reading a file means retrieving data stored in the file.

File Write

Writing a file means storing data into a file.

File Append

Appending means adding new data to the end of an existing file without removing old data.

File Close

Closing a file means terminating the connection between the program and the file.

CONCLUSION

Python provides simple and powerful file handling functions such as `open()`, `read()`, `write()`, `append()`, and `close()`. These operations help programmers store and manage data permanently in files.

FILE OPERATIONS IN PYTHON

Introduction

File operations are used to create, read, write, append, and close files. Python provides several built-in functions for handling files efficiently.

1. OPENING FILES

Definition

Opening a file means establishing a connection between the program and the file.

Syntax

```
file = open(filename, mode)
```

Example

```
file = open("data.txt", "r")
```

Explanation

- data.txt is the file name.
- "r" indicates read mode.
- The file is opened for reading.

Advantages

1. Allows access to external data.
2. Supports different file modes.
3. Enables reading and writing operations.

Disadvantages

1. Wrong mode selection may cause errors.
2. Invalid file paths may raise FileNotFoundError.
3. Forgetting to close files may waste system resources.

2. FILE MODES

Mode Meaning		Description
r	Read	Opens file for reading
w	Write	Overwrites existing content or creates a new file
a	Append	Adds content at the end of the file
r+	Read + Write	Opens file for both reading and writing
b	Binary	Combined with other modes
t	Text	Default mode for text files

Examples

```
open("data.txt", "r")
```

```
open("student.txt", "w")
```

```
open("image.jpg", "rb")
```

Advantages

1. Flexible file access.
2. Supports text and binary files.
3. Allows controlled file manipulation.

Disadvantages

1. Incorrect mode selection can overwrite data.
2. Wrong mode may produce runtime errors.
3. Binary files are difficult to edit manually.

3. READING FILES

Definition

Reading files means retrieving data stored in a file.

Methods Used

read()

Reads the entire file content.

```
file = open("sample.txt", "r")
```

```
data = file.read()
```

```
print(data)
```

```
file.close()
```

Output

Hello Python

Welcome to File Handling

readline()

Reads one line at a time.

```
file = open("sample.txt", "r")
```

```
line1 = file.readline()
```

```
line2 = file.readline()
```

```
print(line1)
```

```
print(line2)
```

```
file.close()
```

Output

Hello Python

Welcome to File Handling

readlines()

Reads all lines and returns a list.

```
file = open("sample.txt", "r")
```

```
lines = file.readlines()
```

```
print(lines)
```

```
file.close()
```

Output

```
['Hello Python\n', 'Welcome to File Handling']
```

Iterating Through a File

```
file = open("sample.txt", "r")
```

```
for line in file:
```

```
    print(line)
```

```
file.close()
```

Output

Hello Python

Welcome to File Handling

Advantages of Reading Files

1. Allows reading full or partial content.
2. Efficient for line-by-line processing.
3. Works with both text and binary files.

Disadvantages of Reading Files

1. Large files may consume more memory.
 2. Incorrect encoding may cause errors.
 3. Improper reading methods may reduce performance.
-

4. WRITING FILES

Definition

Writing files means storing data into a file.

write()

Writes a string to a file.

```
file = open("output.txt", "w")
```

```
file.write("Hello Python")
```

```
file.close()
```

Output in File

Hello Python

writelines()

Writes multiple lines at once.

```
file = open("output.txt", "w")
```

```
file.writelines([
```

```
"Line 1\n",  
"Line 2\n",  
"Line 3\n"  
])
```

```
file.close()
```

Output in File

```
Line 1  
Line 2  
Line 3
```

Advantages of Writing Files

1. Saves user data permanently.
2. Useful for reports and logs.
3. Supports text and binary files.

Disadvantages of Writing Files

1. Existing data may be overwritten.
2. Incorrect encoding may corrupt data.
3. Frequent writing may affect performance.

5. APPENDING FILES

Definition

Appending means adding data to the end of an existing file without removing old data.

Example

```
file = open("sample.txt", "a")
```

```
file.write("\nThis line was added later.")
```

```
file.close()
```

Output in File

Hello Python

Welcome to File Handling

This line was added later.

Advantages

1. Existing data remains safe.
2. New information can be added easily.
3. Useful for logs and records.

Disadvantages

1. File size continuously increases.
2. Duplicate data may occur.
3. Difficult to modify existing records.

6. CLOSING FILES

Definition

Closing a file releases system resources and ensures data is saved properly.

Using close()

```
file = open("sample.txt", "r")
```

```
data = file.read()
```

```
file.close()
```

Using with Statement (Recommended)

```
with open("sample.txt", "r") as file:
```

```
    data = file.read()
```

```
    print(data)
```

Explanation

- File is automatically closed.
- No need to call close() manually.

- Safer and recommended method.

Advantages

1. Frees system resources.
2. Ensures data is saved properly.
3. Prevents memory leaks.
4. with statement automatically closes files.

Disadvantages

1. Forgetting to close files may lock resources.
2. Improper handling can cause data loss.
3. Manual closing requires extra care.

COMBINED FILE HANDLING PROGRAM

Writing

```
file = open("example.txt", "w")
file.write("Hello, this is a test file.\n")
file.write("Python file handling is easy!")
file.close()
```

Reading

```
file = open("example.txt", "r")
print(file.read())
file.close()
```

Appending

```
file = open("example.txt", "a")
file.write("\nThis line was added later.")
```

```
file.close()
```

```
# Reading Again
```

```
file = open("example.txt", "r")
```

```
print(file.read())
```

```
file.close()
```

Output

Hello, this is a test file.

Python file handling is easy!

Hello, this is a test file.

Python file handling is easy!

This line was added later.

WORKING WITH CSV FILES IN PYTHON

Introduction

CSV stands for **Comma Separated Values**.

A CSV file is a plain text file used to store tabular data in rows and columns. Each value in a row is separated by a comma. CSV files are commonly used for storing and exchanging data between applications such as Microsoft Excel, Google Sheets, databases, and Python programs.

Python provides a built-in module called **csv** for reading and writing CSV files efficiently.

What is a CSV File?

A CSV file stores data in a tabular format.

Example: students.csv

Name, Age, Course

Ravi,21,BSc

Anita,20,BCA

Rahul,22,BCom

The above CSV file represents:

Name Age Course

Ravi 21 BSc

Anita 20 BCA

Rahul 22 BCom

Why Use CSV Files?

1. Easy to create and maintain.
2. Human-readable format.
3. Supported by Excel and databases.
4. Easy to exchange data between systems.
5. Widely used in Data Analytics and Machine Learning.

CSV Module

To work with CSV files in Python, import the csv module.

```
import csv
```

Advantages of CSV Files

1. Simple and lightweight format.
2. Easy to read and write.
3. Compatible with Excel, databases, and data science tools.
4. Suitable for data exchange between applications.
5. Supported by many programming languages.

Disadvantages of CSV Files

1. Does not support complex or nested data structures.

2. Sensitive to commas, quotes, and formatting errors.
3. No built-in security mechanism.
4. Large CSV files may be slow to process.
5. Cannot store multimedia data such as images and videos.

READING CSV FILES USING `csv.reader()`

Definition

The `csv.reader()` function is used to read data from a CSV file. Each row is returned as a list.

Program

```
import csv
```

```
with open("data.csv", "r") as f:
```

```
    reader = csv.reader(f)
```

```
    for row in reader:
```

```
        print(row)
```

Sample CSV File

```
Name,Age
```

```
Ravi,21
```

```
Anita,20
```

Output

```
['Name', 'Age']
```

```
['Ravi', '21']
```

```
['Anita', '20']
```

Explanation

Step 1:

```
import csv
```

Imports the CSV module.

Step 2:

```
with open("data.csv", "r") as f:
```

Opens the CSV file in read mode.

Step 3:

```
reader = csv.reader(f)
```

Creates a CSV reader object.

Step 4:

```
for row in reader:
```

Reads each row one by one.

Step 5:

```
print(row)
```

Displays each row as a list.

WRITING CSV FILES USING `csv.writer()`

Definition

The `csv.writer()` function is used to write data into a CSV file.

Program

```
import csv
```

```
with open("output.csv", "w", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    writer.writerow(["Name", "Age"])
```

```
    writer.writerow(["Ravi", 21])
```

```
    writer.writerow(["Anita", 20])
```

Output File

Name, Age

Ravi, 21

Anita,20

Explanation

Step 1:

```
writer = csv.writer(f)
```

Creates a writer object.

Step 2:

```
writer.writerow(["Name", "Age"])
```

Writes the header row.

Step 3:

```
writer.writerow(["Ravi", 21])
```

Writes a data record.

READING CSV FILES USING DictReader

Definition

DictReader reads each row as a dictionary.

The first row of the CSV file is automatically used as dictionary keys.

Program

```
import csv
```

```
with open("students.csv", "r") as f:
```

```
    reader = csv.DictReader(f)
```

```
    for row in reader:
```

```
        print(row)
```

Output

```
{'Name': 'Ravi', 'Age': '21'}
```

```
{'Name': 'Anita', 'Age': '20'}
```

Explanation

Step 1:

```
reader = csv.DictReader(f)
```

Creates a DictReader object.

Step 2:

The header row becomes dictionary keys automatically.

Step 3:

Each row is returned as a dictionary.

Example:

```
{  
'Name': 'Ravi',  
'Age': '21'  
}
```

WRITING CSV FILES USING DictWriter

Definition

DictWriter writes data into a CSV file using dictionaries.

Program

```
import csv
```

```
fields = ["Name", "Age"]
```

```
with open("students.csv", "w", newline="") as f:
```

```
    writer = csv.DictWriter(f, fieldnames=fields)
```

```
    writer.writeheader()
```

```
    writer.writerow({  
        "Name": "Nehal",
```



```
"Age":21  
})
```

```
writer.writerow({  
    "Name":"Srik",  
    "Age":20  
})
```

Output File

```
Name,Age  
Nehal,21  
Srik,20
```

Explanation

Step 1:

```
fields = ["Name", "Age"]
```

Defines the column names.

Step 2:

```
writer = csv.DictWriter(...)
```

Creates a DictWriter object.

Step 3:

```
writer.writeheader()
```

Writes the header row.

Step 4:

```
writer.writerow(...)
```

Writes dictionary data into the CSV file.

Difference Between Reader and DictReader

csv.reader()

Returns Lists

csv.DictReader()

Returns Dictionaries

csv.reader()

Uses Indexes

Less Readable

Suitable for Simple Data

csv.DictReader()

Uses Column Names

More Readable

Suitable for Structured Data

Difference Between Writer and DictWriter**csv.writer() csv.DictWriter()**

Writes Lists Writes Dictionaries

Uses Positions Uses Field Names

Simple Format Structured Format

Less Flexible More Flexible

Real-Time Example

Consider a college student database.

CSV File:

RollNo,Name,Marks

101,Ravi,85

102,Anita,90

103,Rahul,78

Python can:

1. Read student records.
2. Add new students.
3. Update marks.
4. Generate reports.
5. Export data to Excel.

Frequently Used CSV Functions**Function****Purpose**

Function	Purpose
<code>csv.reader()</code>	Reads CSV rows as lists
<code>csv.writer()</code>	Writes CSV rows as lists
<code>csv.DictReader()</code>	Reads CSV rows as dictionaries
<code>csv.DictWriter()</code>	Writes CSV rows as dictionaries
<code>writer.writerow()</code>	Writes one row
<code>writer.writeheader()</code>	Writes column headers

Conclusion

CSV (Comma Separated Values) files are widely used for storing tabular data. Python provides the `csv` module with functions such as `csv.reader()`, `csv.writer()`, `csv.DictReader()`, and `csv.DictWriter()` to efficiently read and write CSV files. CSV files are simple, portable, and commonly used in spreadsheets, databases, reporting systems, and data analysis applications.

OS AND PATHLIB MODULES IN PYTHON

Introduction

Python provides powerful modules for interacting with the operating system, files, folders, and file paths.

The two most important modules are:

1. OS Module
2. Pathlib Module

These modules help programmers perform operations such as:

- Finding the current directory
- Listing files and folders
- Creating directories
- Deleting files
- Checking file existence

- Reading and writing files

1. OS MODULE

Definition

The OS Module is a built-in Python library that provides functions for interacting directly with the operating system.

The OS module works mainly with **string-based file paths**.

Example

```
import os
```

```
file_path = "sample.txt"
```

Here, the path is stored as a string.

Key Points

- Built-in Python module.
- Traditional approach for file management.
- Uses string-based paths.
- Supports file and directory operations.
- Works closely with the operating system.

2. PATHLIB MODULE

Definition

The Pathlib Module is a modern object-oriented module introduced in Python 3.4 for handling file system paths.

Instead of treating paths as strings, Pathlib treats them as Path objects.

Example

```
from pathlib import Path
```

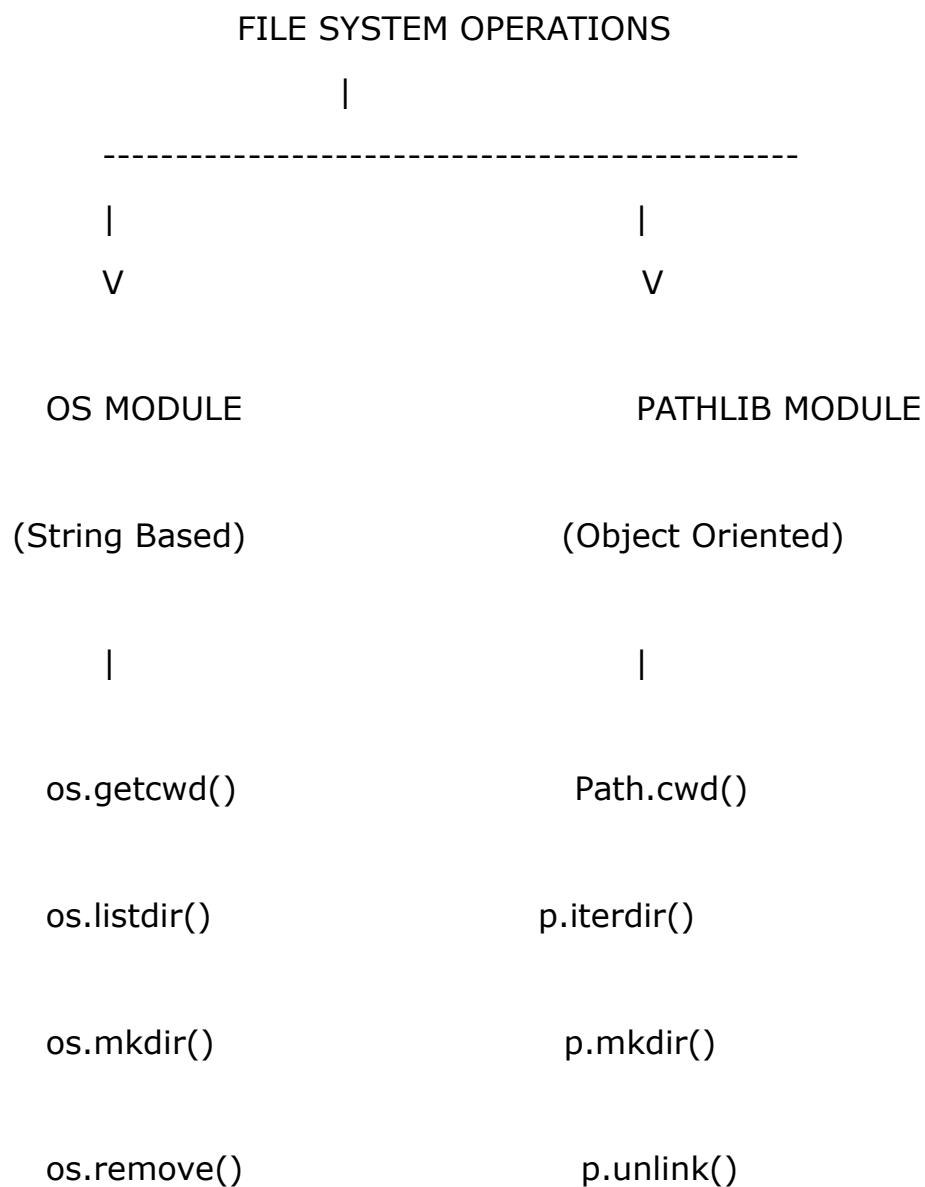
```
p = Path("sample.txt")
```

Here, p is a Path object.

Key Points

- Introduced in Python 3.4.
 - Object-oriented approach.
 - Cleaner and more readable syntax.
 - Platform-independent path handling.
 - Combines features of `os` and `os.path`.
-

Architecture Diagram



os.path.exists()

p.exists()

OS MODULE VS PATHLIB MODULE

Feature	OS Module Pathlib Module	
Approach	Traditional	Modern
Path Representation	String	Path Object
Readability	Moderate	High
Code Length	More	Less
Platform Independence	Limited	Better
Programming Style	Functional	Object-Oriented

Common OS Module Functions

Function	Purpose
os.getcwd()	Current directory
os.listdir()	List files and folders
os.mkdir()	Create folder
os.remove()	Delete file
os.path.exists()	Check file existence

os.getcwd()

Definition

Returns the current working directory.

Program

```
import os
```

```
print(os.getcwd())
```

Output

```
C:\Users\Student
```

Explanation

- getcwd means Get Current Working Directory.
 - Shows the location where Python is currently running.
-

os.listdir()

Definition

Lists all files and folders present in the current directory.

Program

```
import os
```

```
print(os.listdir())
```

Output

```
['data.txt', 'student.csv', 'Images']
```

Explanation

Returns a list containing all files and folders.

os.mkdir()

Definition

Creates a new directory.

Program

```
import os
```

```
os.mkdir("newdir")
```

Explanation

Creates a folder named newdir.

os.path.exists()

Definition

Checks whether a file or folder exists.

Program

```
import os
```

```
print(os.path.exists("sample.txt"))
```

Output

True

os.remove()

Definition

Deletes a file permanently.

Program

```
import os
```

```
os.remove("sample.txt")
```

Complete OS Module Program

```
import os
```

```
print("Current Directory:", os.getcwd())
```

```
print("Directory Contents:", os.listdir())
```

```
if not os.path.exists("newdir"):
    os.mkdir("newdir")
```

```
with open("sample.txt", "w") as f:
    f.write("Hello from Python")
```

```
print(os.path.exists("sample.txt"))
```



```
os.remove("sample.txt")
```

```
print("sample.txt deleted")
```

Advantages of OS Module

1. Direct interaction with operating system.
2. Powerful file and directory management.
3. Useful for automation scripts.
4. Supports low-level system operations.

Disadvantages of OS Module

1. Uses lengthy syntax.
2. Requires os.path for path operations.
3. Less readable for beginners.

Important Pathlib Functions

Function	Purpose
-----------------	----------------

Path()	Create path object
exists()	Check existence
read_text()	Read file
write_text()	Write file
unlink()	Delete file
cwd()	Current directory
iterdir()	List directory contents

Path().write_text()

Definition

Writes text directly into a file.

Program

```
from pathlib import Path
```

```
p = Path("example.txt")
```

```
p.write_text("Hello Python")
```

Path().read_text()

Definition

Reads the entire content of a file.

Program

```
from pathlib import Path
```

```
p = Path("example.txt")
```

```
print(p.read_text())
```

Output

```
Hello Python
```

Path().exists()

Definition

Checks whether the file exists.

Program

```
from pathlib import Path
```

```
p = Path("example.txt")
```

```
print(p.exists())
```

Output

True

Path().unlink()

Definition

Deletes a file.

Program

```
from pathlib import Path
```

```
p = Path("example.txt")
```

```
p.unlink()
```

Complete Pathlib Program

```
from pathlib import Path
```

```
p = Path("example.txt")
```

```
p.write_text("This file is created using pathlib.")
```

```
if p.exists():
```

```
    print("example.txt exists")
```

```
print("Content of example.txt:")
```

```
print(p.read_text())
```

```
p.unlink()
```

```
print("example.txt deleted")
```

Output

example.txt exists

Content of example.txt:

This file is created using pathlib.

example.txt deleted

Advantages of Pathlib Module

1. Cleaner syntax.
2. Easy to understand.
3. Object-oriented approach.
4. Platform-independent paths.
5. Combines os and os.path features.
6. Provides high-level file operations.

Disadvantages of Pathlib Module

1. Requires learning object-oriented concepts.
2. Less common in older programs.
3. Some advanced system tasks still require OS module.

Real-Time Example

In a College Management System:

- Create folders for departments.
- Store student records.
- Verify file existence.
- Read student reports.
- Delete old records.

These operations can be performed efficiently using OS and Pathlib modules.

Quick Revision Diagram

OS MODULE

getcwd() -> Current Directory

listdir() -> List Files

mkdir() -> Create Folder

exists() -> Check File

remove() -> Delete File

PATHLIB MODULE

Path() -> Create Path Object

exists() -> Check File

write_text() -> Write File

read_text() -> Read File

unlink() -> Delete File

Exam Definitions

OS Module

"The OS module is a built-in Python module that provides functions for interacting with the operating system and managing files and directories."

Pathlib Module

"The Pathlib module is a modern object-oriented module used for handling file paths and performing file operations."

Conclusion

The OS Module and Pathlib Module are essential tools for file and directory management in Python. The OS module uses a traditional string-based approach, whereas Pathlib provides a modern object-oriented approach.

Pathlib is generally preferred because of its cleaner syntax, readability, and platform-independent path handling.

EXCEPTION HANDLING AND ASSERTIONS IN PYTHON

Complete Master Notes for B.Sc., BCA & Degree Students

UNIT OVERVIEW

This unit covers:

1. Error Types in File Handling
2. Exception Handling
3. try-except Block
4. Multiple except Blocks
5. Generic except Block
6. try-except-else Block
7. try-except-finally Block
8. raise Statement
9. User-Defined Exceptions
10. Assertions
11. Common Exception Types

1. INTRODUCTION TO EXCEPTION HANDLING

Definition

Exception Handling is a mechanism used to detect, handle, and recover from runtime errors without terminating the program abruptly.

Why Exception Handling?

- Prevents program crashes
- Improves reliability
- Helps debugging
- Handles unexpected situations gracefully
- Provides user-friendly error messages

EXCEPTION HANDLING ARCHITECTURE

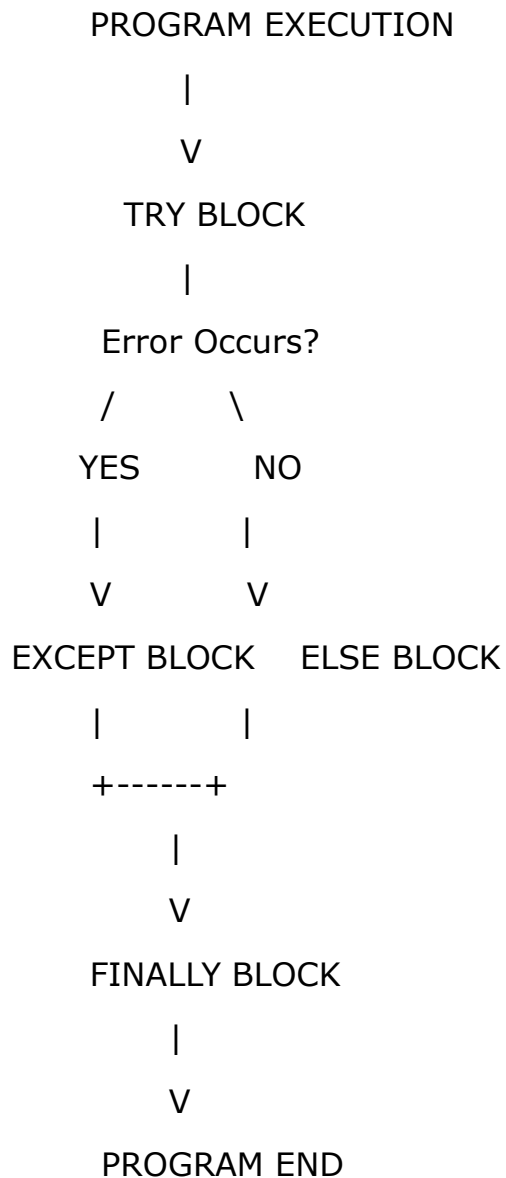


Diagram Explanation

1. Program enters the try block.
 2. Python monitors for runtime errors.
 3. If an error occurs, control moves to the matching except block.
 4. If no error occurs, else block executes.
 5. Finally block always executes.
 6. Program terminates normally.
-

2. ERROR TYPES IN FILE HANDLING

FileNotFoundError

Definition

Occurs when a file does not exist.

Example

```
open("missing.txt")
```

Output

FileNotFoundError

PermissionError

Definition

Occurs when the user lacks permission to access a file.

Example

```
open("protected.txt","w")
```

IsADirectoryError

Definition

Occurs when a directory is opened as a file.

Example

```
open("Documents")
```

IOError / OSError

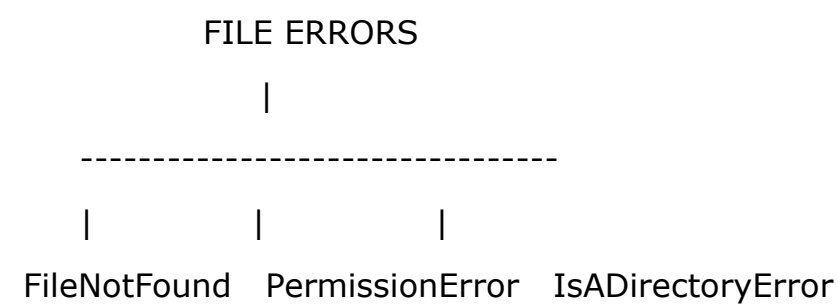
Definition

General input-output related errors.

Examples

- Hardware failures
 - Disk errors
 - Invalid file operations
-

FILE ERROR DIAGRAM



3. TRY-EXCEPT BLOCK

Definition

Used to handle exceptions safely.

Syntax

try:

 risky code

except ErrorType:

 handling code

Example

try:

 f = open("data.txt", "r")

except FileNotFoundError:

 print("File not found")

Output

File not found

Advantages

- Prevents crashes
- Improves robustness
- Easy error recovery

Disadvantages

- Excessive use reduces readability

4. MULTIPLE EXCEPT BLOCKS

Definition

Handles different exceptions separately.

Syntax

try:

code

except Error1:

code

except Error2:

code

Example

try:

```
num = int(input("Enter number: "))  
print(10/num)
```

except ZeroDivisionError:

```
print("Cannot divide by zero")
```

except ValueError:

```
print("Invalid input")
```

5. GENERIC EXCEPT BLOCK

Definition

Handles unknown exceptions.

Syntax

try:

code

```
except Exception as e:  
    print(e)
```

Example

```
try:  
    a = 10/0
```

```
except Exception as e:  
    print(e)
```

Output

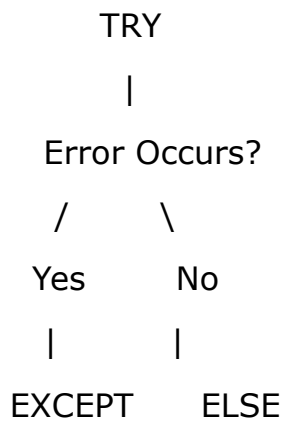
division by zero

6. TRY-EXCEPT-ELSE BLOCK

Definition

Else executes only when no exception occurs.

Architecture Diagram



Example

```
try:  
    result = int("100")
```

```
except ValueError:
```

```
print("Conversion Failed")
```

else:

```
print("Conversion Successful")
```

Output

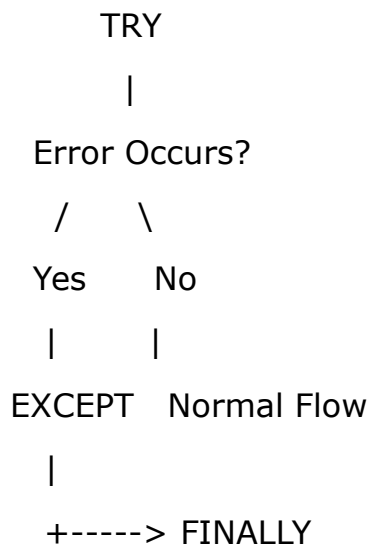
Conversion Successful

7. TRY-EXCEPT-FINALLY BLOCK

Definition

Finally executes whether an exception occurs or not.

Architecture Diagram



Example

try:

```
file = open("sample.txt")
```

except FileNotFoundError:

```
print("File Missing")
```

finally:

```
print("Cleanup Complete")
```

8. RAISE STATEMENT

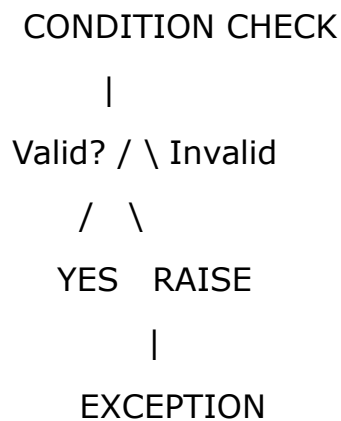
Definition

Used to manually generate exceptions.

Syntax

```
raise ExceptionType("Error Message")
```

Architecture Diagram



Example

```
age = -5
```

```
if age < 0:  
    raise ValueError("Age cannot be negative")
```

Output

```
ValueError: Age cannot be negative
```

9. USER-DEFINED EXCEPTIONS

Definition

Custom exceptions created by inheriting from Exception.

Syntax

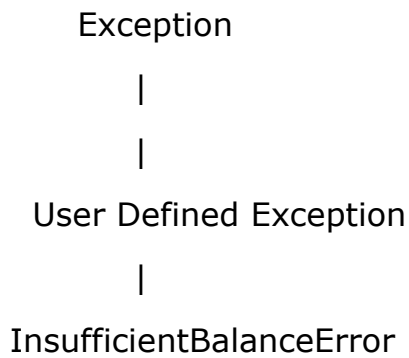
```
class MyError(Exception):  
    pass
```

Example

```
class InsufficientBalanceError(Exception):  
    pass
```

```
raise InsufficientBalanceError("Balance Too Low")
```

Architecture Diagram



Applications

- Banking Systems
- Inventory Systems
- Authentication Systems

10. ASSERTIONS

Definition

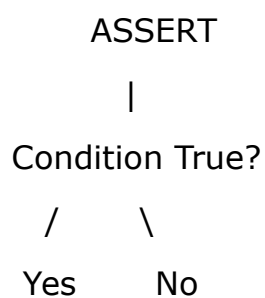
Assertions are debugging statements used to verify whether a condition is true.

If the condition is false, Python raises an AssertionError.

Syntax

```
assert condition, "Error Message"
```

ASSERTION ARCHITECTURE



V	V

Continue AssertionError

Example Program

```
x = 10
```

```
assert x > 0, "x must be positive"
```

```
print("Assertion Passed")
```

Output

Assertion Passed

Example Program (Failure)

```
y = -5
```

```
assert y >= 0, "y must not be negative"
```

Output

AssertionError:

y must not be negative

Uses of Assertions

- Debugging
 - Testing assumptions
 - Detecting logical errors
 - Verifying program conditions
-

Advantages

- Detects bugs early

- Improves code quality
- Acts as internal validation

Disadvantages

- Not suitable for user input validation
- Can terminate program execution

11. COMMON EXCEPTION TYPES

Exception	Description
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Wrong data type
FileNotFoundError	Missing file
IndexError	Index out of range
KeyError	Missing dictionary key
PermissionError	Access denied
AssertionError	Assertion failed

COMPREHENSIVE EXCEPTION HANDLING PROGRAM

try:

```
file = open("data.txt","r")
```

```
num = int(input("Enter number: "))
```

```
assert num > 0, "Number must be positive"
```

```
result = 100 / num
```



```

except FileNotFoundError:
    print("File missing")

except ZeroDivisionError:
    print("Division by zero")

except ValueError:
    print("Invalid input")

except AssertionError as e:
    print(e)

except Exception as e:
    print("Unknown Error:", e)

else:
    print("Result =", result)

finally:
    print("Program execution completed")

```

MASTER COMPARISON TABLE

Feature	try except else finally raise assert					
Detect Error	Yes	No	No	No	No	Yes
Handle Error	No	Yes	No	No	No	No
Success Code	No	No	Yes	No	No	No
Cleanup Tasks	No	No	No	Yes	No	No
Generate Error	No	No	No	No	Yes	Yes

Feature	try	except	else	finally	raise	assert
---------	-----	--------	------	---------	-------	--------

Debugging	No	No	No	No	No	Yes
-----------	----	----	----	----	----	-----

QUICK REVISION

try

Contains risky code.

except

Handles exceptions.

else

Executes when no exception occurs.

finally

Always executes.

raise

Generates exceptions manually.

User-Defined Exception

Custom exception created by programmer.

assert

Checks assumptions during execution.

CONCLUSION

Exception Handling is one of the most important features of Python. It helps programs handle runtime errors gracefully using try, except, else, finally, raise, user-defined exceptions, and assertions. Proper exception handling improves reliability, robustness, maintainability, and user experience.